

第13讲 文件系统(1)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

文件系统概述

02

文件系统的硬件支持

03

文件

04

目录

文件系统概述

- 信息是计算机系统中的重要资源，所有的计算机应用系统都要存储信息和检索信息，操作系统的一个重要功能就是负责信息的组织、存储和访问
- 存储信息的三个基本要求：
 - 能够存储大量的信息
 - 长期保存信息
 - 可以共享信息
- 解决方法：把信息以一种单元——即文件的形式存储在磁盘或其他存储介质上

文件系统概述

- 文件是通过操作系统来管理的。文件的结构、命名、存取、使用、保护和实现方法都是操作系统设计的主要内容
- 操作系统中处理文件的部分称为文件系统，它管理文件的存储、检索、更新，提供安全可靠的共享和保护手段，并且方便用户使用

文件系统的功能

- **方便的文件访问和控制**：以符号名称作为文件标识，便于用户使用
- **并发文件访问和控制**：在多道程序系统中支持对文件的并发访问和控制
- **统一的用户接口**：在不同设备上提供同样的接口，方便用户操作和编程
- **多种文件访问权限**：在多用户系统中的不同用户对同一文件会有不同的访问权限
- **优化性能**：存储效率、检索性能、读写性能
- **差错恢复**：能够验证文件的正确性，并具有一定的差错恢复能力

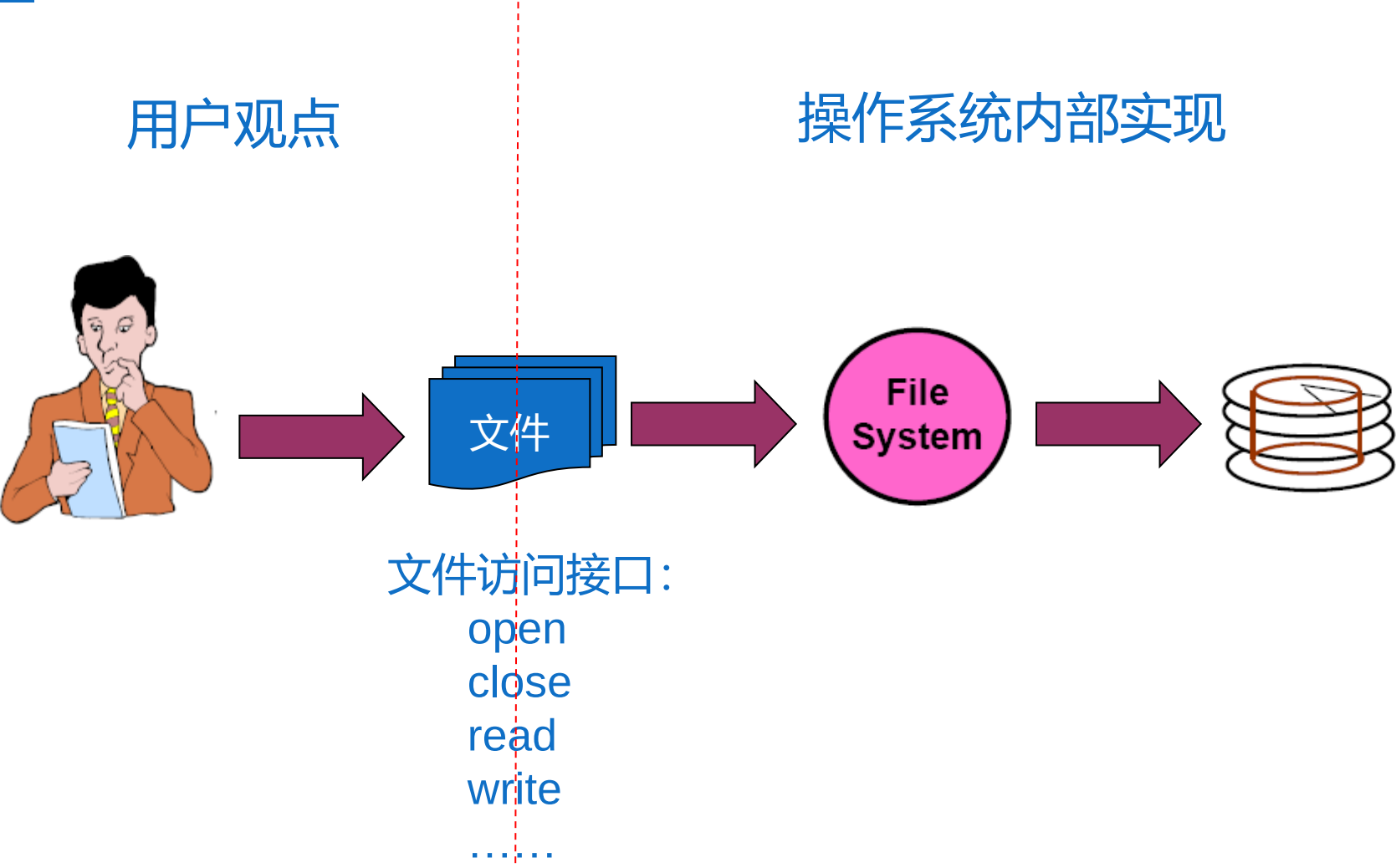
文件系统的执行效率

- 对大多数用户而言，文件系统是操作系统中最为可见的部分
 - 操作系统和所有用户的程序与数据的存储都是由文件系统负责的
- 文件系统在操作系统接口中占的比例最大，用户使用操作系统的感觉在很大程度上取决于对文件系统的使用效果

文件系统的两种观点

- 用户观点：文件系统如何呈现在用户面前——一个文件由什么组成，如何命名，如何保护文件，可以进行何种操作等等
- 操作系统观点：文件和目录怎样实现，怎样管理存储空间，文件存储位置，磁盘实际运作方式(与设备管理的接口)，文件系统性能等等

文件系统的两种观点



从用户的观点看，操作系统中引入文件系统的主要目的是：

- ☐ A 保护用户数据
- ☐ B 实现虚拟存储器
- ☐ C 保存用户和系统的数据
- ☒ D 实现对文件的按名存取

提交

大纲

01

文件系统概述

02

文件系统的硬件支持

03

文件

04

目录

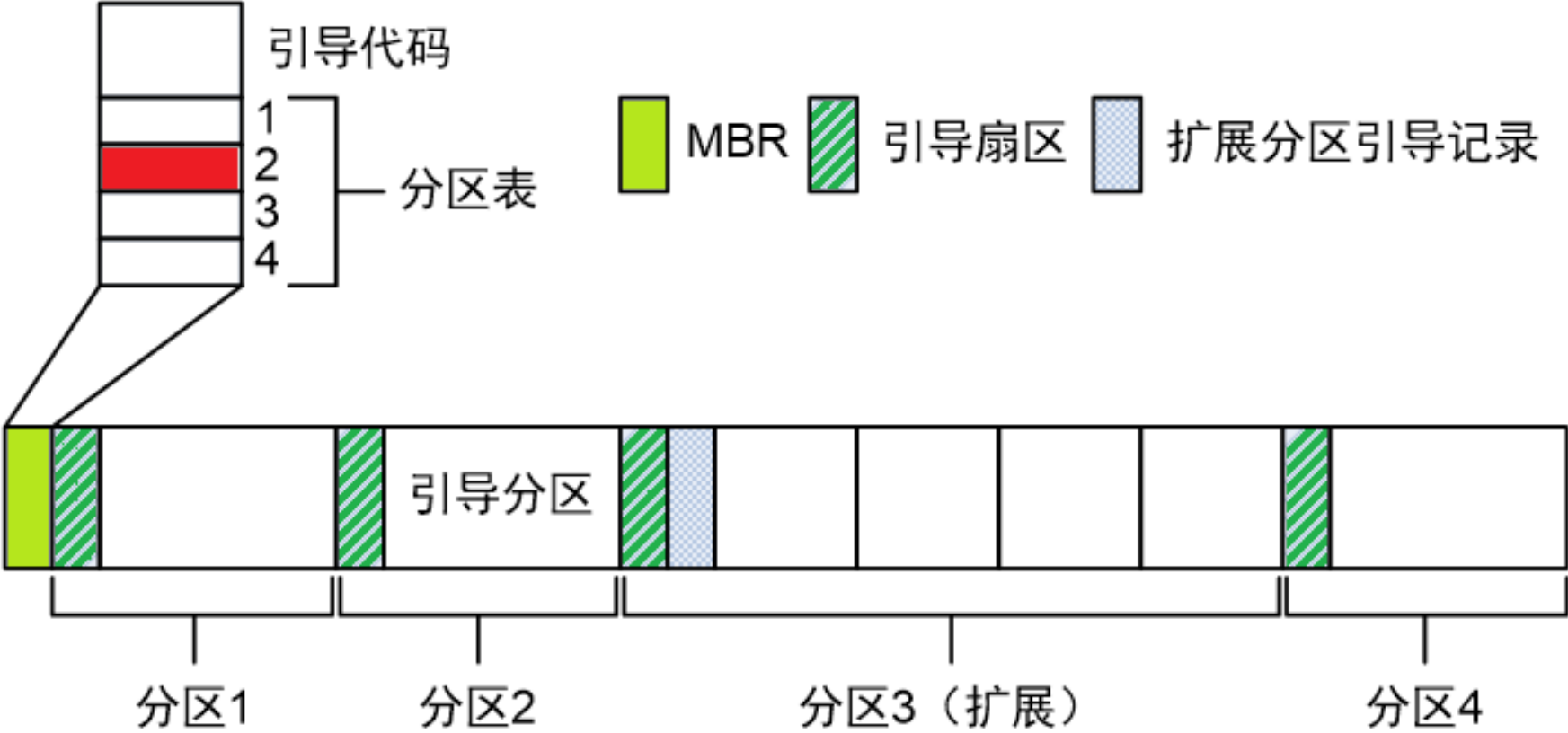
磁盘分区

- 物理磁盘按照扇区(sector)来寻址，扇区的大小由硬件决定，通常是512字节
- 分区(partition)是磁盘上一组连续扇区的集合
- 每个硬盘最多可以包括4个主分区，而每个主分区最多可以分配4个扩展分区，扩展分区又可以包含扩展分区。依此类推，实际在磁盘上可以设置的分区个数没有上限

磁盘分区

- 磁盘的0号扇区称为**主引导记录(MBR)**，用来引导计算机
- MBR大小固定，其中包含了可执行的指令(称为引导代码)和分区表，分区表中给出了每个分区的起始和结束地址，以及指示该分区是否可引导的标记

磁盘分区



磁盘分区示例

主分区

扩展分区

扩展分区中的逻辑分区

可引导标记

```
hbma@ubuntu: ~  
hbma@ubuntu:~$ sudo fdisk /dev/sda  
[sudo] password for hbma:  
  
Welcome to fdisk (util-linux 2.34).  
Changes will remain in memory only, until you decide to write them.  
Be careful before using the write command.  
  
Command (m for help): p  
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors  
Disk model: VMware Virtual S  
Units: sectors of 1 * 512 = 512 bytes  
Sector size (logical/physical): 512 bytes / 512 bytes  
I/O size (minimum/optimal): 512 bytes / 512 bytes  
Disklabel type: dos  
Disk identifier: 0x91ac5807  
  
Device      Boot  Start      End  Sectors  Size Id Type  
/dev/sda1   *      2048    1050623    1048576    512M  b W95 FAT32  
/dev/sda2           1052670  41940991  40888322    19.5G   5 Extended  
/dev/sda5           1052672  41940991  40888320    19.5G   83 Linux  
  
Command (m for help):
```

磁盘分区示例

转储MBR

```
hbma@ubuntu:~$ sudo dd if=/dev/sda of=mbr.bin bs=512 count=1
1+0 records in
1+0 records out
512 bytes copied, 0.000658347 s, 778 kB/s
hbma@ubuntu:~$
```

```
hbma@ubuntu:~$ hexdump -C mbr.bin
00000000 eb 63 90 10 8e d0 bc 00 b0 b8 00 00 8e d8 8e c0 |.C.....|
00000010 fb be 00 7c bf 00 06 b9 00 02 f3 a4 ea 21 06 00 |...|.....!..|
00000020 00 be be 07 38 04 75 0b 83 c6 10 81 fe fe 07 75 |...8.u.....u|
00000030 f3 eb 16 b4 02 b0 01 bb 00 7c b2 80 8a 74 01 8b |.....|...t..|
00000040 4c 02 cd 13 ea 00 7c 00 00 eb fe 00 00 00 00 00 |L.....|.....|
00000050 00 00 00 00 00 00 00 00 00 00 80 01 00 00 00 00 |.....|.....|
00000060 00 00 00 00 ff fa 90 90 f6 c2 80 74 05 f6 c2 70 |.....t...p|
00000070 74 02 b2 80 ea 79 7c 00 00 31 c0 8e d8 8e d0 bc |t....y|..1.....|
00000080 00 20 fb a0 64 7c 3c ff 74 02 88 c2 52 bb 17 04 |. ..d|<.t...R...|
00000090 f6 07 03 74 06 be 88 7d e8 17 01 be 05 7c b4 41 |...t...}.....|.A|
000000a0 bb aa 55 cd 13 5a 52 72 3d 81 fb 55 aa 75 37 83 |..U..ZRR=..U.u7.|
000000b0 e1 01 74 32 31 c0 89 44 04 40 88 44 ff 89 44 02 |..t21..D.@.D..D.|
000000c0 c7 04 10 00 66 8b 1e 5c 7c 66 89 5c 08 66 8b 1e |....f..\\f.\\f..|
000000d0 60 7c 66 89 5c 0c c7 44 06 00 70 b4 42 cd 13 72 |`|f.\\..D..p.B..r|
000000e0 05 bb 00 70 eb 76 b4 08 cd 13 73 0d 5a 84 d2 0f |...p.v....s.Z...|
000000f0 83 d0 00 be 93 7d e9 82 00 66 0f b6 c6 88 64 ff |.....}...f....d.|
00000100 40 66 89 44 04 0f b6 d1 c1 e2 02 88 e8 88 f4 40 |@f.D.....@|
00000110 89 44 08 0f b6 c2 c0 e8 02 66 89 04 66 a1 60 7c |.D.....f..f.`||
00000120 66 09 c0 75 4e 66 a1 5c 7c 66 31 d2 66 f7 34 88 |f..uNf.\\|f1.f.4.|
00000130 d1 31 d2 66 f7 74 04 3b 44 08 7d 37 fe c1 88 c5 |.1.f.t.;D.}7....|
00000140 30 c0 c1 e8 02 08 c1 88 d0 5a 88 c6 bb 00 70 8e |0.....Z....p.|
00000150 c3 31 db b8 01 02 cd 13 72 1e 8c c3 60 1e b9 00 |.1.....r...`...|
00000160 01 8e db 31 f6 bf 00 80 8e c6 fc f3 a5 1f 61 ff |...1.....a.|
00000170 26 5a 7c be 8e 7d eb 03 be 9d 7d e8 34 00 be a2 |&Z|..}....}.4...|
00000180 7d e8 2e 00 cd 18 eb fe 47 52 55 42 20 00 47 65 |}......GRUB .Ge|
00000190 6f 6d 00 48 61 72 64 20 44 69 73 6b 00 52 65 61 |om.Hard Disk.Rea|
000001a0 64 00 20 45 72 72 6f 72 0d 0a 00 bb 01 00 b4 0e |d. Error.....|
000001b0 cd 10 ac 3c 00 75 f4 c3 07 58 ac 91 00 00 80 04 |...<.u...X.....|
000001c0 01 04 0b fe c2 ff 00 08 00 00 00 00 10 00 00 fe |.....|
000001d0 c2 ff 05 fe c2 ff fe 0f 10 00 02 e8 6f 02 00 00 |.....o...|
000001e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
000001f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa |.....U.|
00000200
hbma@ubuntu:~$
```

磁盘分区示例

```
hbma@ubuntu: ~  
hbma@ubuntu:~$ ndisasm mbr.bin | cat  
00000000 EB63      jmp short 0x65  
00000002 90          nop  
00000003 10ED0BC     adc [bp-0x4330],cl  
00000007 00B0B800   add [bx+si+0xb8],dh  
0000000B 008ED88E   add [bp-0x7128],cl  
0000000F C0FBBE     sar bl,byte 0xbe  
00000012 007CBF     add [si-0x41],bh  
00000015 0006B900   add [0xb9],al  
00000019 02F3       add dh,bl  
0000001B A4         movsb  
0000001C EA21060000 jmp 0x0:0x621  
00000021 BEBE07     mov si,0x7be  
00000024 3804       cmp [si],al  
00000026 750B       jnz 0x33  
00000028 83C610     add si,byte +0x10  
0000002B 81FEFE07   cmp si,0x7fe  
0000002F 75F3       jnz 0x24  
00000031 EB16       jmp short 0x49  
00000033 B402       mov ah,0x2  
00000035 B001       mov al,0x1  
00000037 BB007C     mov bx,0x7c00  
0000003A B280       mov dl,0x80  
0000003C 8A7401     mov dh,[si+0x1]  
0000003F 8B4C02     mov cx,[si+0x2]
```

```
hbma@ubuntu: ~  
00000065 FA         cli  
00000066 90         nop  
00000067 90         nop  
00000068 F6C280     test dl,0x80  
0000006B 7405       jz 0x72  
0000006D F6C270     test dl,0x70  
00000070 7402       jz 0x74  
00000072 B280       mov dl,0x80  
00000074 EA797C0000 jmp 0x0:0x7c79  
00000079 31C0       xor ax,ax  
0000007B 8ED8       mov ds,ax  
0000007D 8ED0       mov ss,ax  
0000007F BC0020     mov sp,0x2000  
00000082 FB         sti  
00000083 A0647C     mov al,[0x7c64]  
00000086 3CFF       cmp al,0xff  
00000088 7402       jz 0x8c  
0000008A 88C2       mov dl,al  
0000008C 52         push dx  
0000008D BB1704     mov bx,0x417  
00000090 F60703     test byte [bx],0x3  
00000093 7406       jz 0x9b  
00000095 BE887D     mov si,0x7d88  
00000098 E81701     call 0x1b2  
0000009B BE057C     mov si,0x7c05
```

PC机的系统引导过程

- PC机引导时首先执行BIOS，由BIOS选择一个引导设备，将该设备的MBR读入内存，然后将控制权交给MBR的引导代码
- 引导代码扫描主分区表，直到找到引导分区(boot partition)
- MBR的引导代码将引导分区中的引导扇区(boot sector)读入内存，并将控制权交给引导扇区代码。引导扇区是引导分区中的第一个扇区

GPT分区

- MBR分区支持的存储容量上限为2TB，为支持更大容量的磁盘引入了GTP分区
- GPT=GUID Partition Table(GUID分区表)
- GPT分区的存储容量理论上无限制；MBR分区最多支持四个主分区，GPT分区没有限制
- 为了兼容，GPT分区也有MBR引导，称为保护引导

MBR分区 & GPT分区

←

≡

设置

🔍

系统 > 存储 > 磁盘和卷 > Lexar SSD ARES 4TB

状态

联机

驱动器运行状况

估计剩余生命: 100%

可用备用: 100%

温度: 60°C

单击此处获取详细信息

立即备份

分区样式

GUID 分区表(GPT)

获取帮助

SSD硬盘

←

≡

设置

🔍

系统 > 存储 > 磁盘和卷 > Kingston DataTraveler 3.0

○

性能

使用写入缓存来提高磁盘性能。为防止潜在数据丢失，请在断开设备连接之前使用“安全删除硬件并弹出媒体”图标。

状态

联机

脱机

分离

分离 VHD

分区样式

主启动记录(MBR)

获取帮助

USB盘

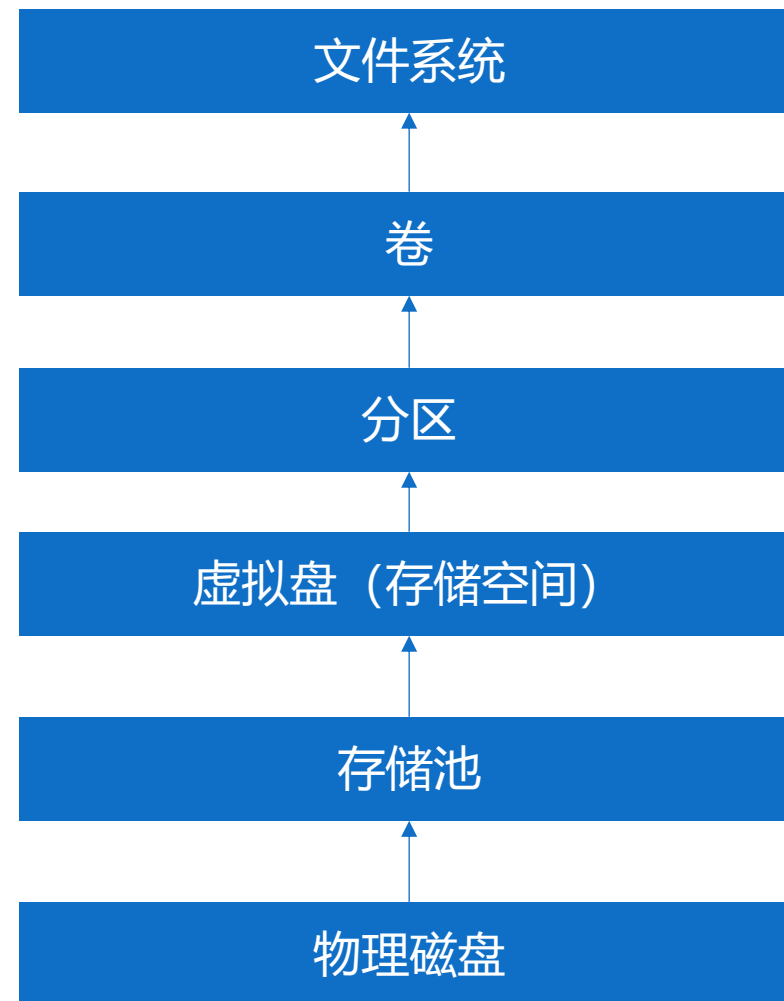
Windows的存储体系

■技术演进

时代	核心技术	技术特点
Windows NT ~ XP	基本盘 (Basic Disk)	■ 物理磁盘上创建分区 ■ 分区上创建 文件系统
Windows 2000 ~ Windows 10	动态盘 (Dynamic Disk)	■ 引入卷管理 ■ 使用LDM(逻辑磁盘管理)执行磁盘和卷管理 ■ 可以调整动态磁盘的大小而无需重新启动操作系统
Windows 11	存储空间 (Storage Spaces)	■ 引入存储虚拟化

Windows 11的存储空间架构

- 物理磁盘实际的存储设备，包括HDD、SSD
- 多个物理磁盘组成统一的存储池，存储池负责统一管理磁盘、元数据管理、空间分配、故障状态跟踪、数据布局
- 从存储池中切出虚拟盘，虚拟盘是一个逻辑块设备（Logical Block Device），支持冗余策略（包括简单、镜像、奇偶）
- 虚拟盘上可以创建分区、进行格式化和挂载
- 卷是已经格式化、可供文件系统使用的逻辑空间



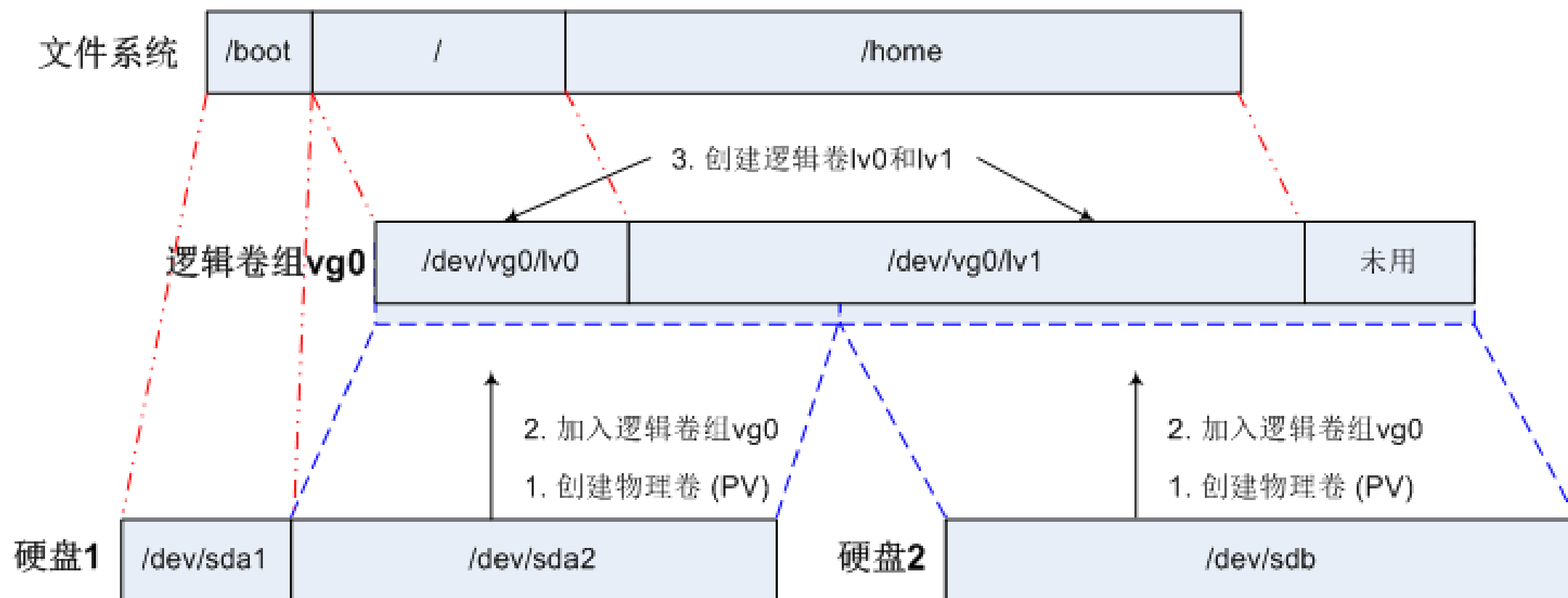
Windows 11的IO驱动栈



Windows 11支持的存储空间类型

存储空间	特点	对应传统 RAID
简单(Simple)	条带化，无冗余	RAID 0
双路镜像(Two-way mirror)	双副本镜像	RAID 1
三路镜像(Three-way mirror)	三副本镜像	对RAID 1的增强
奇偶(Parity)	单奇偶校验	RAID 5
双奇偶(Dual parity)	双奇偶校验	RAID 5

Linux的逻辑卷管理



文件系统格式

- 文件系统在文件卷上创建
- 文件系统格式(file system format)定义了文件数据在文件卷上的存储方式，它影响了一个文件系统的各种特性

文件系统格式

■ FAT文件系统格式

引导区	文件分配表1	文件分配表2	根目录	其他目录和文件
-----	--------	--------	-----	---------

- FAT表——两个镜像，互为备份。文件卷中的每个簇均对应一个FAT表项，文件分配采用链式分配方法

文件系统格式

■ Unix类操作系统的文件系统格式

Boot block	Super block	Free space mgmt	I-nodes	Root dir	Files and directories
------------	-------------	-----------------	---------	----------	-----------------------

- 超级块——包含文件系统的所有关键参数
- 空闲块信息
- i节点——一个数据结构数组，每个元素对应一个文件

文件系统有关的术语

■ 簇(cluster)

- 是文件系统格式使用的可寻址数据块，簇的大小总是扇区大小的整数倍
- 文件系统使用簇来管理磁盘空间
- 簇越大，其潜在缺点是可能会产生内部碎片，浪费更多的磁盘空间

■ 元数据(metadata)

- 是为支持文件系统格式管理而存储在卷上的数据。例如，元数据包含了卷上的文件和目录的位置信息等
- 对于应用程序，元数据通常是不可访问的

关于卷管理，以下描述**错误**的是

- ☐ A 卷提高了物理磁盘的可靠性和利用率
- ☐ B 卷是建立在硬盘之上的一个逻辑层
- ☒ C 引入卷提高了文件系统的安全性
- ☐ D 卷为文件系统屏蔽下层磁盘分区的布局

提交

大纲

01

文件系统概述

02

文件系统的硬件支持

03

文件

04

目录

文件

- 文件是一种抽象机制，它提供了一种在磁盘上保存信息并方便以后读取的方法。这种方法可以使用户不用了解存储信息的方法、位置和实际磁盘工作方式等有关细节
- **文件**：一组具有标识符号的在逻辑上具有完整意义的信息项的集合，这个标识符号称为文件名

文件

- 文件包括两部分
- 文件体：文件本身的信息
- 文件属性：文件的存储和管理信息，例如：文件名、文件内部标识、文件存储地址、访问权限、访问时间等

文件属性

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

文件命名

- 文件的命名规则在不同的操作系统中是不同的
 - 长度
 - 数字和特殊字符
 - 大小写区分
 - 文件扩展名

文件扩展名

- 文件扩展名通常用于提示文件内容、类型等相关信息
- Linux/UNIX中，文件扩展名只是一种约定，操作系统并不强迫采用它
- Windows对扩展名赋予含义，可以在操作系统中注册扩展名，并且规定哪个程序打开该文件

文件扩展名

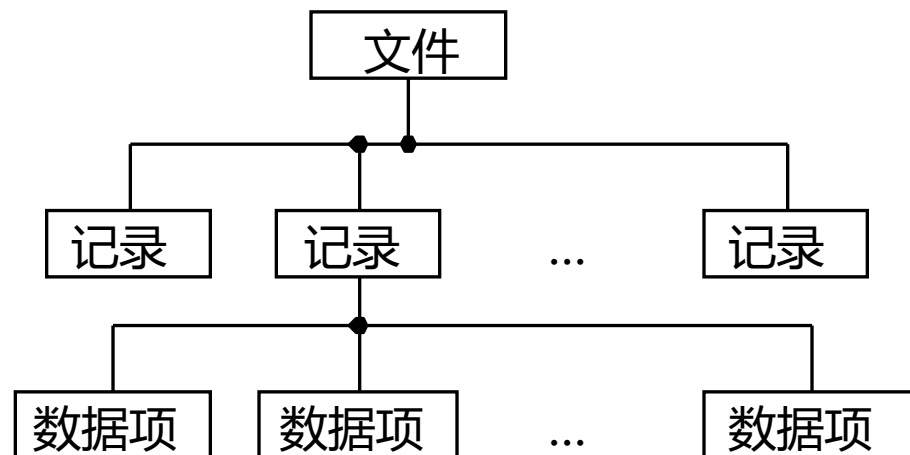
Extension	Meaning
file.bak	Backup file
file.c	C source program
file.gif	Compuserve Graphical Interchange Format image
file.hlp	Help file
file.html	World Wide Web HyperText Markup Language document
file.jpg	Still picture encoded with the JPEG standard
file.mp3	Music encoded in MPEG layer 3 audio format
file.mpg	Movie encoded with the MPEG standard
file.o	Object file (compiler output, not yet linked)
file.pdf	Portable Document Format file
file.ps	PostScript file
file.tex	Input for the TEX formatting program
file.txt	General text file
file.zip	Compressed archive

文件的逻辑结构

- 从用户角度看文件，研究文件的组织形式，它可以独立于在外存上的物理存储
- 文件逻辑结构的设计要求：
 - 访问性能：便于检索；便于修改
 - 存储性能：向物理存储转换方便，节省空间
 - 文件的不同组织层次：域(数据项)、记录、文件

文件的逻辑结构

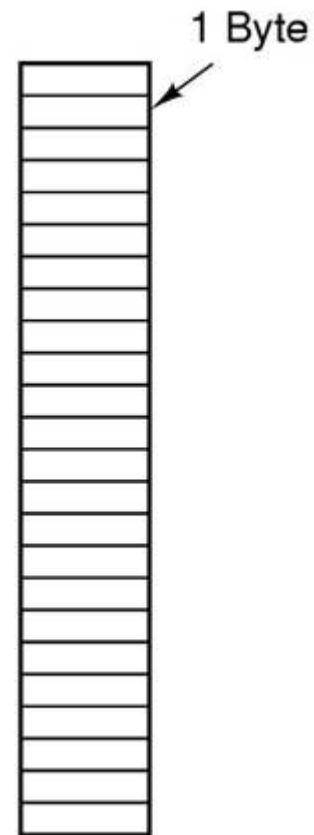
■ 文件的不同组织层次



文件的逻辑结构

■无结构文件

- 文件体为字节流，不划分记录，构成文件的基本单位是字节
- 优点：提供最大的灵活性



文件的逻辑结构

- 累积文件(pile)
- 文件体为无结构记录序列，通过特定分隔符来划分记录，各记录大小和组成可变，新记录总是添加到文件末尾
- 如日志log，或电子邮件的邮箱文件(mailbox)

文件的逻辑结构

- 顺序文件(sequential file)

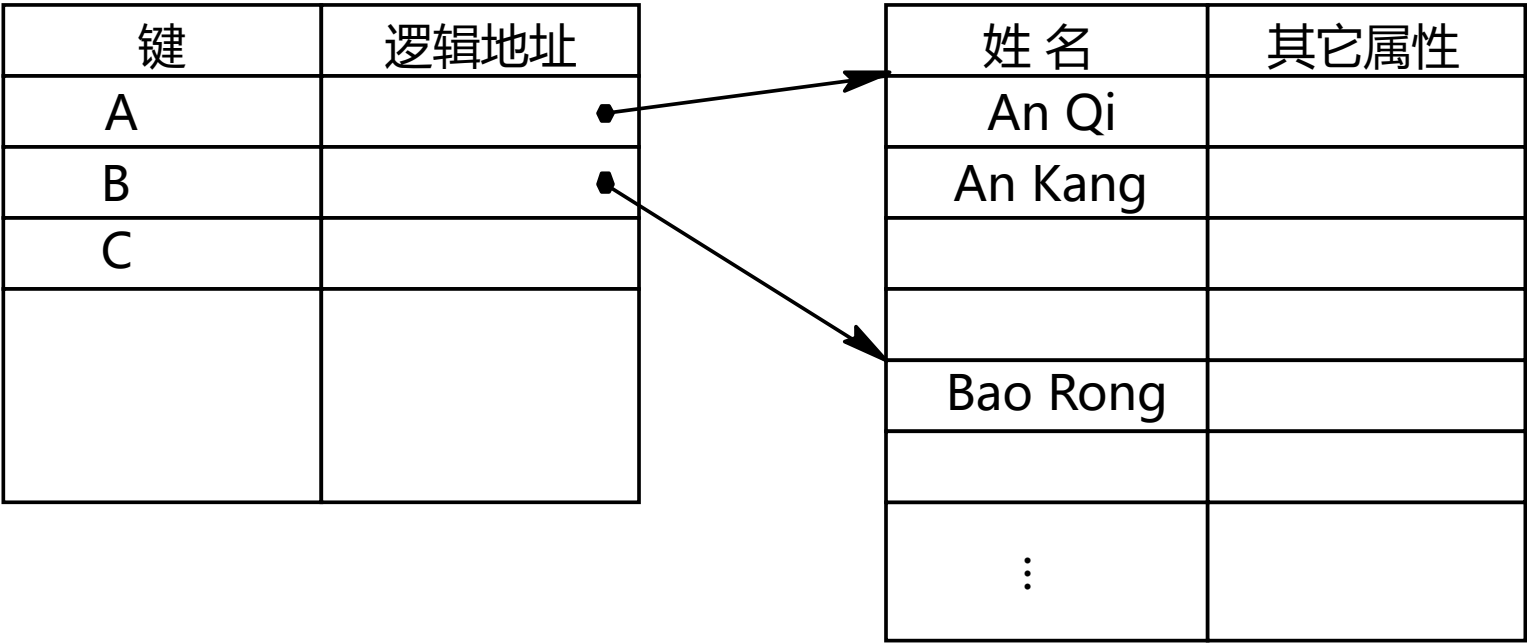
- 文件体为大小相同的排序记录序列。它由一个主文件和一个临时文件组成。记录大小相同，按某个关键字域(key field)排序，存放在主文件(master file)中
- 新记录暂时保存在日志或事务文件(log file or transaction file)中，定期归并入主文件

文件的逻辑结构

- 索引顺序文件(indexed-sequential file)
- 在顺序文件(主文件main file)的基础上，另外建立索引(index)和溢出文件(overflow file)，目的是加快顺序文件的检索速度
- 在索引文件中，可将关键字域中的取值划分若干个区间，每个区间对应一个索引项，后者指向该区间的开头记录
- 新记录暂时保存在溢出文件中，定期归并入主文件

文件的逻辑结构

索引顺序文件(indexed-sequential file)



顺序文件

文件的逻辑结构

- 索引文件(indexed file)
- 记录大小不必相同，不必排序，存放在主文件(primary file)中
- 索引文件与索引顺序文件的区别在于主文件不排序，另外建立索引，每个索引项指向一个记录，索引项按照记录中的某个关键字域排序
- 对同一主文件，可以针对不同的关键字域相应建立多个索引。索引文件的记录项通常较小，查找速度快，便于随机访问(random access)

文件的逻辑结构

- 哈希文件或直接文件(hashed file or direct file)
- 记录大小相同。由主文件和溢出文件组成。记录位置由哈希函数确定
- 检索时给出记录编号，通过哈希函数计算出该记录在文件中的相对位置
- 访问速度快，但在主文件中有空闲空间

文件类型

■按文件性质和用途分类

- 系统文件：与操作系统有关的文件
- 用户文件
- 库文件：标准子程序及常用应用程序组成文件，允许用户使用但不能修改

■按信息保存期限分类

- 临时文件；永久文件；档案文件

文件类型

- 按文件的保护方式分类

- 只读文件；读写文件；可执行文件

- 按文件的逻辑结构分类

- 流式文件(字节序列)；记录式文件

- 按文件的物理结构分类

- 顺序(连续)文件；链接文件；索引文件

文件类型

■ Linux/UNIX系统将文件分为三类

- 普通文件：包含用户信息，一般为ASCII或二进制文件
- 目录文件：是管理文件系统结构的系统文件
- 特殊文件：设备文件，把外部设备也看作文件。包括字符特殊文件和块特殊文件

文件存取

- **顺序存取**：只能从头开始按顺序存取文件的字节或记录，不能跳过某些内容
- **随机存取**：可以不按顺序存取文件中的字节或记录，或者按照关键字而不是位置来存取记录

文件操作

- 以Linux/UNIX的系统调用为例

- creat**: 创建一个空文件或者将已有的文件截空

`fd=creat(name, pmode);`

fd——返回的文件描述符，调用失败时返回-1

name——创建的文件名

pmode——文件的保护模式

文件操作

- **open**: 打开一个已存在的文件

```
fd = open(name, rwmode[, pmode]);
```

rwmode——文件的读写模式

- O_RDONLY——只读
- O_WRONLY——只写
- O_RDWR——读写

- **close**: 关闭一个已打开的文件

```
close(fd);
```

文件操作

■ read/write: 对文件进行读写操作

`n = read(fd, buffer, size);`

`n = write(fd, buffer, size);`

`n`——读写操作返回的字节数

`buffer`——指向读写缓冲器的指针

`size`——请求读写的字节数

文件操作

■ lseek: 改变文件指针所指的位置

`newpos= lseek(fd, offset, origin);`

`newpos`——若lseek调用成功返回文件指针新位置，否则返回-1

`offset`——表示从指定位置开始的偏移量

`origin`——确定文件指针偏移量的基准点

- `SEEK_SET`——从文件首开始
- `SEEK_CUR`——从文件当前位置开始
- `SEEK_END`——从文件尾开始

open系统调用的目的是

- ☐ A 把文件的内容从辅存读到内存
- ☒ B 把文件的属性信息从辅存读到内存
- ☐ C 把文件系统的控制信息从辅存读到内存
- ☐ D 把文件的索引信息从辅存读到内存

提交

文件的逻辑结构是由_____决定的。

- ☐ A 存储介质特性
- ☐ B 内存容量
- ☐ C 操作系统的管理方式
- ☒ D 用户

提交

大纲

01

文件系统概述

02

文件系统的硬件支持

03

文件

04

目录

文件控制块

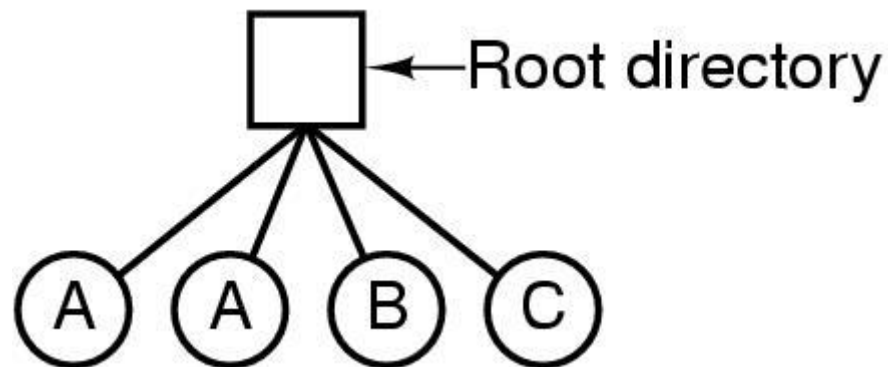
- 文件控制块(File Control Block, FCB)是操作系统为管理文件而设置的数据结构, 它存放了为管理文件所需的所有有关信息(文件属性)
- 文件控制块是文件存在的标志
- 文件控制块的内容:
 - 文件名, 文件号, 用户名, 文件地址, 文件长度, 文件类型, 文件属性, 共享计数, 文件的建立日期, 保存期限, 最后修改日期, 最后访问日期, 口令, 文件逻辑结构, 文件物理结构

目录

- **文件目录**：把所有的FCB组织在一起，就构成了文件目录，即文件控制块的有序集合
- **目录项**：构成文件目录的项目(目录项可以就是FCB)
- **目录文件**：为了实现对文件目录的管理，通常将文件目录以文件的形式保存在外存，该文件称为目录文件

目录结构

- **一级目录系统**：为所有文件建立一个目录文件(组成一线性表)
- 具有如下的特点：
 - 结构简单，易实现
 - 文件多时，目录检索时间长
 - 限制了用户对文件的命名



目录结构

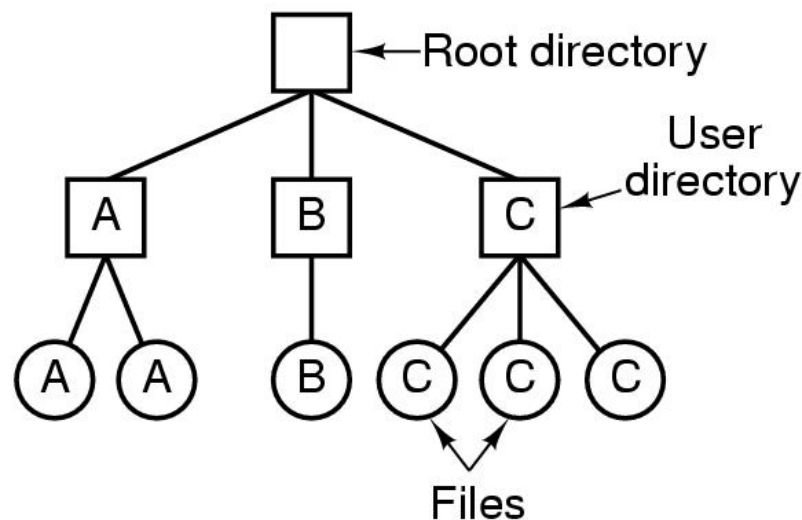
■两级目录系统：目录分为两级

- 一级称为主文件目录，给出用户名，用户子目录所在的物理位置
- 二级称为用户文件目录(又称用户子目录)，给出该用户所有文件的FCB

■优点：

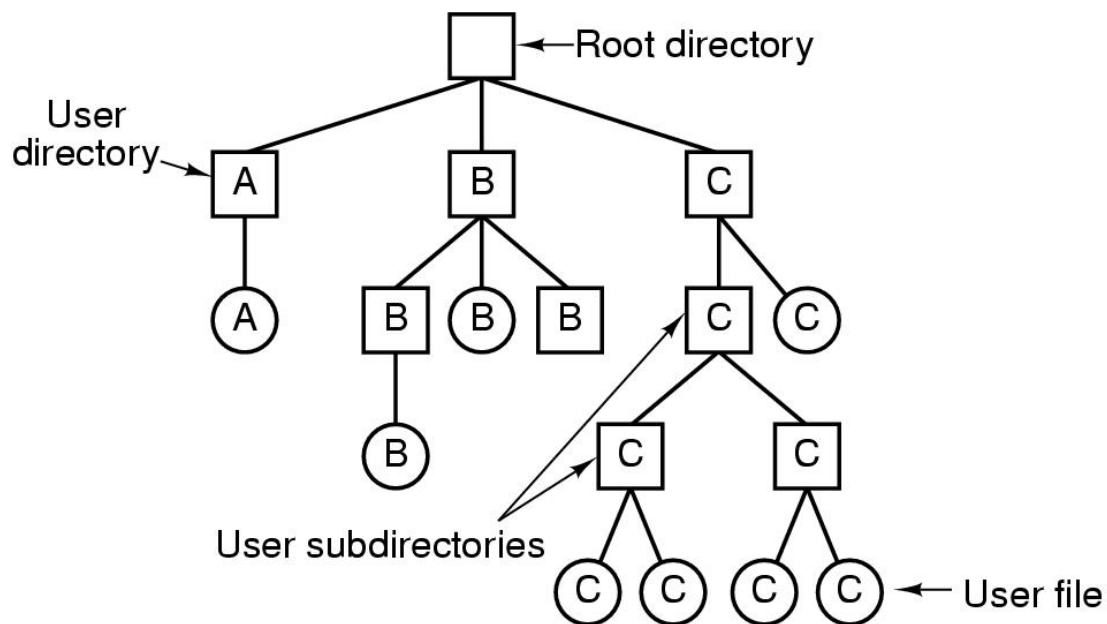
- 解决了文件的重名问题
- 查找时间降低

■缺点：增加了系统开销



目录结构

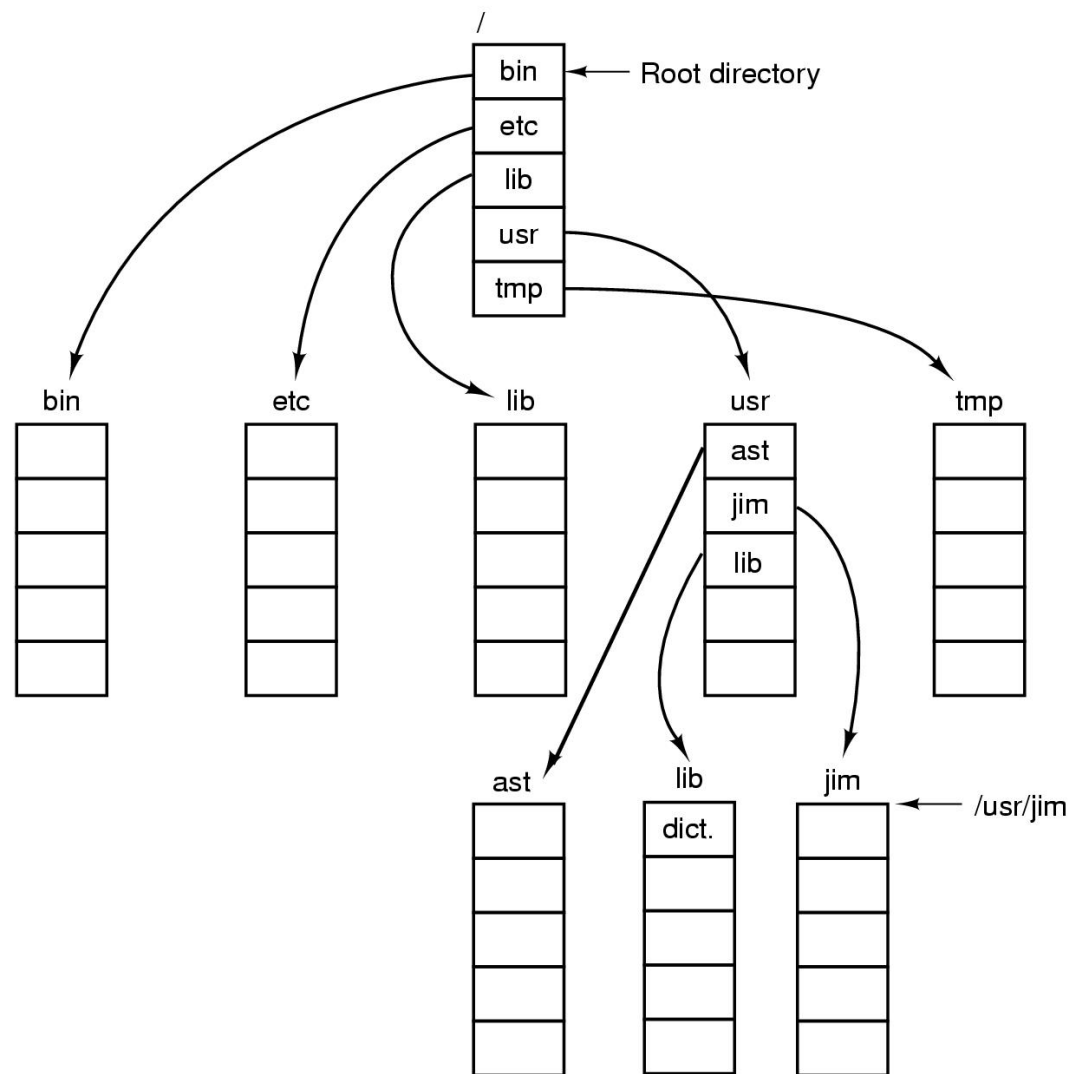
- **层次目录系统：**也称为树状目录。在文件数目较多时，便于系统和用户将文件分散管理
- 适用于较大的文件系统管理



路径名

- 用目录树组织文件系统时，需要有某种方法指明文件名
- **绝对路径名**：由从根目录到文件的路径组成
- **相对路径名**：用户可以指定一个目录为当前工作目录，这时，所有不从根目录开始的路径名都是相对于当前工作目录的
- "."——指当前目录
- ".."——指当前目录的父目录

路径名



目录操作

- 以Linux/UNIX的系统调用为例

- mkdir**: 创建一个目录

`int mkdir(const char *path, mode_t mode);`

返回值: 调用失败时返回-1, 否则返回0

参数:

path——创建的目录路径名

mode——目录的保护模式

目录操作

■ rmdir: 删除一个目录

`int rmdir(const char *path);`

返回值: 调用失败时返回-1, 否则返回0

参数:

path——待删除的目录路径名

目录操作

■ `opendir`: 打开一个目录

`DIR *opendir(const char *dirname);`

返回值: 返回指向DIR类型的指针, 调用失败时返回NULL

参数:

dirname——目录名

目录操作

■closedir: 关闭一个目录

`int closedir(DIR *dirp);`

返回值: 调用失败时返回-1, 否则返回0

参数:

dirp——opendir返回的指针

目录操作

■readdir: 读取一个目录项

`struct dirent *readdir(DIR *dirp);`

返回值: 指向dirent结构的指针, 调用失败时返回-1

参数:

dirp——opendir返回的指针

目录操作

■link: 创建一个文件的连接

`int link(const char *existing, const char *new);`

返回值: 调用失败时返回-1, 否则返回0

参数:

existing——现有文件的路径名

new——链接文件的路径名

目录操作

■ unlink: 删除目录项

`int unlink(const char *path);`

返回值: 调用失败时返回-1, 否则返回0

参数:

path——要删除的文件路径名

在文件系统中引入当前目录的主要目的是_____。

- ☒ A 提高系统的性能
- ☐ B 增强系统的安全性
- ☐ C 支持共享访问
- ☒ D 方便用户使用

提交

操作系统，2026

谢谢

OPERATING SYSTEM